

Machine Learning: Overview

CS115 - Math for Computer Science

September 9, 2024

What is Machine Learning?

- ML is an active subfield of AI
- ML seek to answer the question

How can we build computer systems that automatically improve with experience, and what are the fundamental laws that govern learning process?

Mitchell, 2016

What is machine learning?

*A computer program is said to learn from **experience E** with respect to some class of **tasks T**, and **performance measure P**, if its performance at tasks in **T**, as measured by **P**, improves with experience **E**.*

— Tom Mitchell

There are many different kinds of ML, depending on:

- ✓ the task **T** we wish the system to learn
- ✓ the performance measure **P** we use to evaluate the system
- ✓ the training signal or experience **E** we give it

Improve on task T , with respect to
performance metric P , based on experience E

Spam filtering for emails

T: filter/predict/classify all emails that are spam

P: The accuracy of prediction, that is the percentage of emails
that are correctly classified into normal/spam

E: Set of emails, each with a label of spam/normal

What is machine learning?

Improve on task T , with respect to
performance metric P , based on experience E

Hand-written words Recognition

T: Recognizing hand-written words

P: Percentage of words correctly classified

E: Database of human-labeled images of handwritten words

What is machine learning?

Image tagging/annotation/captioning



T: Give some words that describe the meaning of a picture

P: ???

E: Set of picture, each has been captioned with a set of words

Evaluation (P)

- ☐ Confusion matrix
- ☐ Accuracy
- ☐ Precision
- ☐ Recall
- ☐ F-score
- ☐ ROC
- ☐ AUC
- ☐ IoU
- ☐ Explained Variance
- ☐

Evaluation (P)

$$\text{accuracy} = \frac{\text{correct predictions}}{\text{all predictions}}$$

$$\text{precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

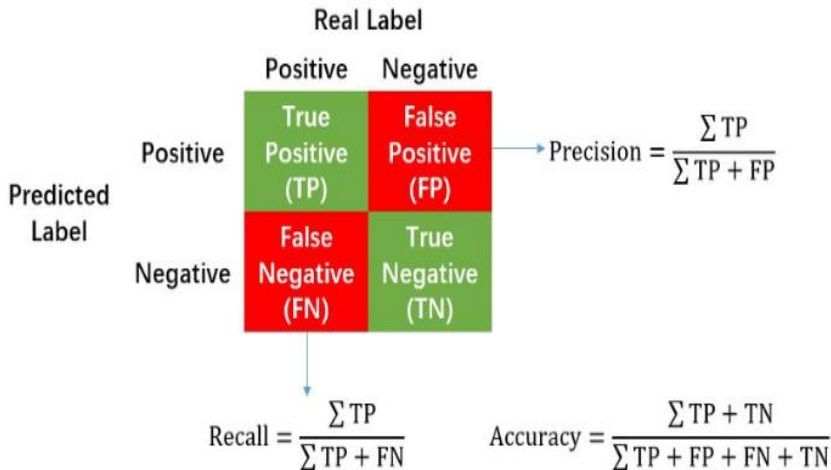
$$\text{recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

$$F_{\beta} = (1 + \beta^2) \frac{\text{Precision} * \text{Recall}}{(\beta^2 * \text{Precision}) + \text{Recall}}$$

$$F1 = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

		Actual Value (as confirmed by experiment)	
		positives	negatives
Predicted Value (predicted by the test)	positives	TP True Positive	FP False Positive
	negatives	FN False Negative	TN True Negative

Evaluation (P)



What is machine learning?

Some more examples of tasks that are best solved by using a learning algorithm

- Recognizing patterns:
 - Facial identities or facial expressions
 - Handwritten or spoken words
 - Medical images
- Generating patterns:
 - Generating images or motion sequences
- Recognizing anomalies:
 - Unusual credit card transactions
 - Unusual patterns of sensor readings in a nuclear power plant
- Prediction:
 - Future stock prices or currency exchange rates

What does a machine learn?

- A mapping (function):

$$f : x \mapsto y$$

- x: observations (data), past experience
- y: prediction, new knowledge, new experience,...

A Model

- ✓ Data are often **supposed** to follow or be generated from an unknown model
- ✓ Learning a model means learning the **parameters** of that model

What is machine learning?

Where does a machine learn from?

Learn from a set of training samples

After learning?

- ✓ We obtain a model, new knowledge, or new experience
- ✓ We can use that model/function to do prediction or inference for future observation

What is machine learning?

MACHINE LEARNING

Elements of Machine Learning

1

Generalization:

- ❖ Generalize from specific examples
- ❖ Based on statistical inference

2

Data:

- ❖ Training data: specific examples to learn from
- ❖ Test data: (new) specific examples to assess performance

3

Models:

- ❖ Theoretical assumptions about the task/domain
- ❖ Parameters that can be inferred from data

4

Algorithms:

- ❖ Learning algorithm: infer model (parameters) from data
- ❖ Inference algorithm: infer predictions from model

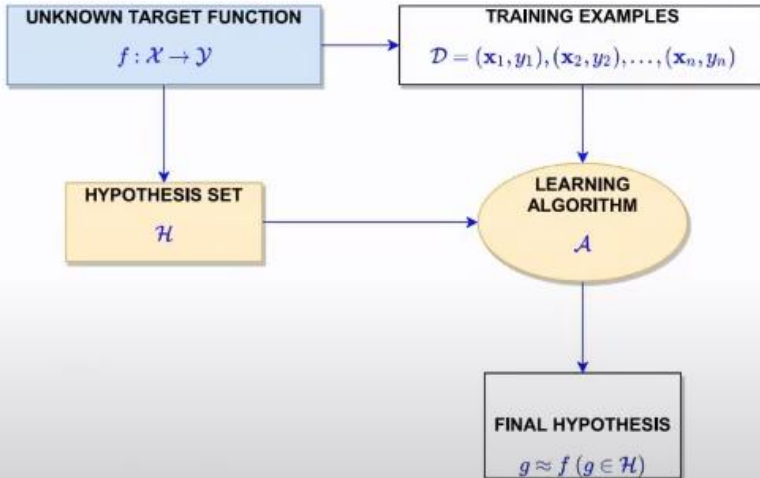
Learning Components

Notations

symbol	meaning
$a, b, c, N \dots$	scalar number
$\mathbf{w}, \mathbf{v}, \mathbf{x}, \mathbf{y} \dots$	column vector
$\mathbf{X}, \mathbf{Y} \dots$	matrix
$\mathbb{R}$	set of real numbers
$\mathbb{Z}$	set of integer numbers
$\mathbb{N}$	set of natural numbers
$\mathbb{R}^D$	set of vectors
$\mathcal{X}, \mathcal{Y}, \dots$	set
$\mathcal{A}$	algorithm

operator	meaning
$\mathbf{w}^T$	transpose
$\mathbf{X}\mathbf{Y}$	matrix multiplication
$\mathbf{X}^{-1}$	inverse

Learning Components



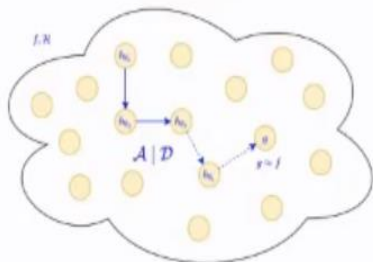
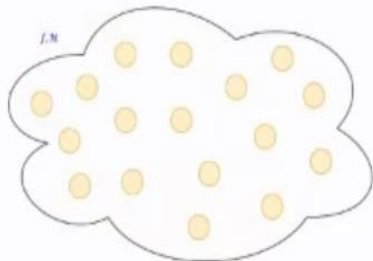
The two components are referred as the **learning model**

- The **hypothesis set** $\mathcal{H}$ is a set of functions that is potentially similar to f

$$\mathcal{H} = \{h_{\theta_1}, h_{\theta_2}, \dots\}$$

- The **learning algorithm** $\mathcal{A}$ is a **search algorithm** which finds $g \in \mathcal{H}$ such that

$$g \overset{\text{best}}{\approx} f$$

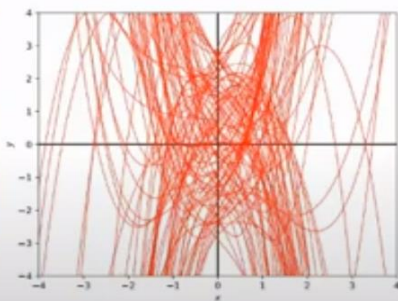
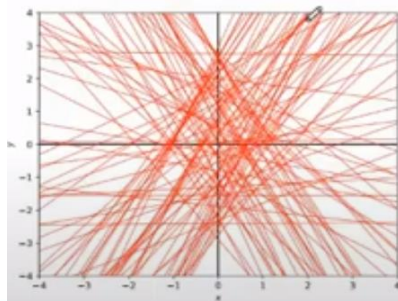


ML COMPONENTS

Hypothesis set

- Hypothesis set is a set of **potential** functions, models or solutions.

- Hypothesis set can be **infinite**. For example, sets of functions $y = \theta_0 + \theta_1 x$ and $y = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$



Examples: Credits Approval

Suppose that a bank receives thousands of card applications every day and its wants to automate the process of evaluating them

Age: 23 years

Gender: male

Annual salary: \$30000

Year of residence: 1 year

Years in job: 1 year

Current debt: \$15000

Approve credit?

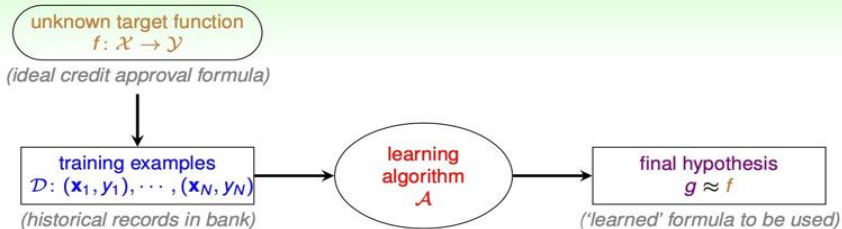
Problem Statement

Formalization

- Input: $\mathbf{x}$ (*customer application*)
- Output: y (*good/bad customer? or $\{1, -1\}$*)
- Data $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)$ (*historical records*)
- Target function: $f : \mathcal{X} \rightarrow \mathcal{Y}$ (*ideal credit approval formula*)
- Best approximate function $g : \mathcal{X} \rightarrow \mathcal{Y}$ (*formula to be used*)

Problem Statement

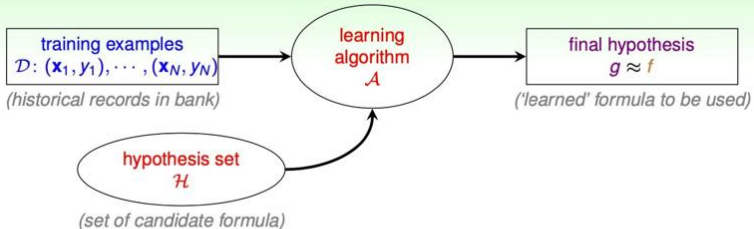
Learning Flow for Credit Approval



- target f **unknown**
(i.e. no programmable definition)
- hypothesis g hopefully $\approx f$
but possibly **different** from f
(perfection ‘impossible’ when f unknown)

What does g look like?

The Learning Model



- assume $g \in \mathcal{H} = \{h_k\}$, i.e. approving if
 - h_1 : annual salary > NTD 800,000
 - h_2 : debt > NTD 100,000 (really?)
 - h_3 : year in job ≤ 2 (really?)
- hypothesis set $\mathcal{H}$:
 - can contain **good or bad hypotheses**
 - up to $\mathcal{A}$ to pick the 'best' one as g

learning model = $\mathcal{A}$ and $\mathcal{H}$

A Simple Hypothesis Set-Example

The Perceptron model

- For input $\mathbf{x} = (x_1, \dots, x_d)$ (*attributes of a customer*)

Approve credit if $\sum_{i=1}^d w_i x_i \geq \text{threshold}$

Deny credit if $\sum_{i=1}^d w_i x_i < \text{threshold}$

- This linear formula $h \in \mathcal{H}$ can be written as

$$h(\mathbf{x}) = h_{\mathbf{w}, \text{threshold}}(\mathbf{x}) = \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) - \text{threshold} \right)$$

A Simple Hypothesis Set-Example

The Perceptron model

- Set $w_0 = -\text{threshold}$

$$h(x) = h_w(x) = \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) + w_0 \right)$$

- Introduce an artificial coordinate $x_0 = 1$

$$\mathcal{H}(x) = h_w(x) = \text{sign} \left(\sum_{i=0}^d w_i x_i \right)$$

- In vector form, the perceptron implements

$$h(x) = h_w(x) = \text{sign} (w^T x)$$

A Simple Hypothesis Set-Example

The Perceptron model- A Simple Learning Algorithm

- Given the training set

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$$

1. Init $\mathbf{w}$

2. Repeat until satisfied

- At iteration $t = 1, 2, 3, \dots$, pick a *misclassified* point $(\mathbf{x}_i, y_i)$

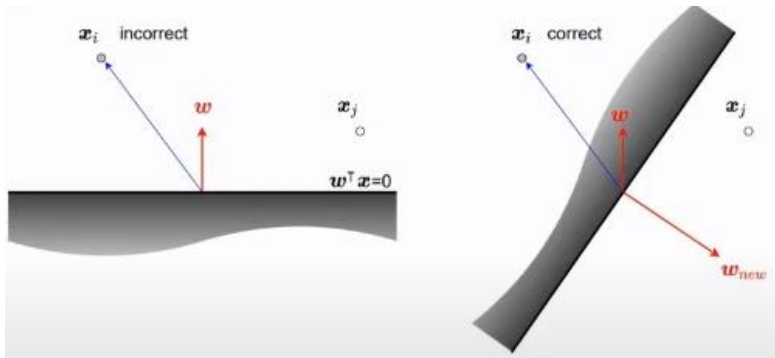
$$\text{sign}(\mathbf{w}^T \mathbf{x}_i) \neq y_i$$

- and update the weight vector

$$\mathbf{w} \leftarrow \mathbf{w} + y_i \mathbf{x}_i$$

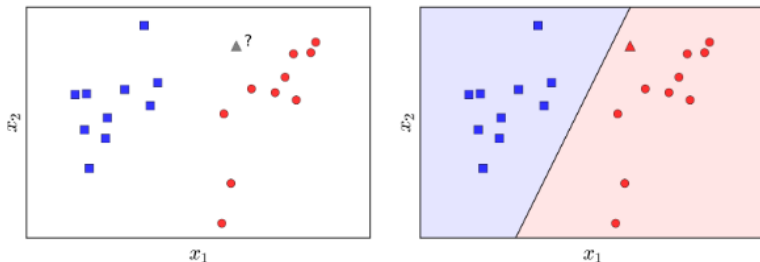
A Simple Hypothesis Set-Example

The Perceptron model- A Simple Learning Algorithm



A Simple Hypothesis Set-Example

The Perceptron model



$$\text{label}(\mathbf{x}) = \text{sgn}(\mathbf{w}^T \mathbf{x})$$

$$\text{label}(\mathbf{x}) = 1 \text{ if } \mathbf{w}^T \mathbf{x} \geq 0, \text{ otherwise } -1$$

ML issues

Learning algorithm

- Under what conditions the chosen algorithm will converge?
- For a given application/domain and a given objective function, what algorithm perform best?

No free lunch Theorem

- All models are wrong, but some models are useful.

— George Box

ML issues

No free lunch theorem

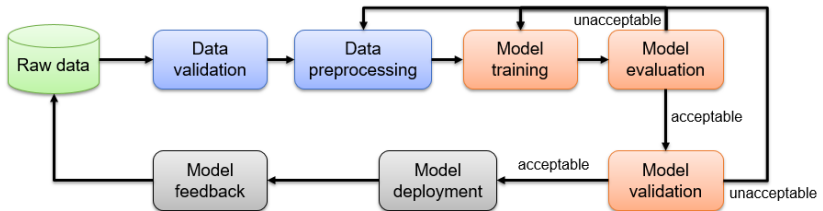
- There is no single best model that works optimally for all kinds of problems.
- A set of assumptions (**inductive bias**) that works well in one domain may work poorly in another.
- We pick a suitable model based on domain knowledge, and/or trial and error (using model selection techniques).
- It is important to have many models and algorithmic techniques to choose from.

Summary

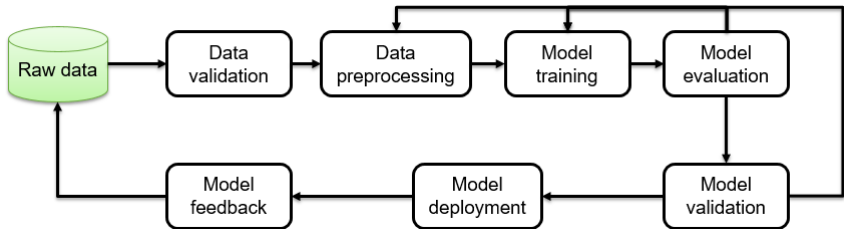
Learning? What do we mean?

- Machine learning involves building mathematical models to help understand data.
- In practice, we use data to compute **hypothesis g** that approximate unknown **target f** .
- In practice, **learning algorithm $\mathcal{A}$** takes training examples **$\mathcal{D}$** and **hypothesis set $\mathcal{H}$** to get **final hypothesis g** .
- "Learning" enters the picture when we give these models tunable parameters that can be adapted to observed data; in this way the program can be considered to be "learning" from the data.

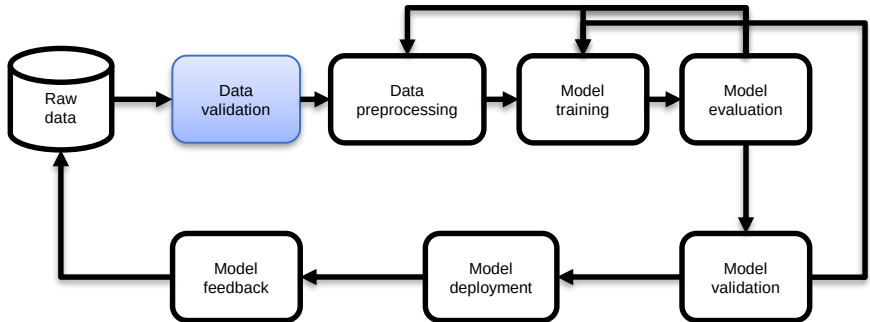
Machine Learning Pipeline



Machine Learning Pipeline

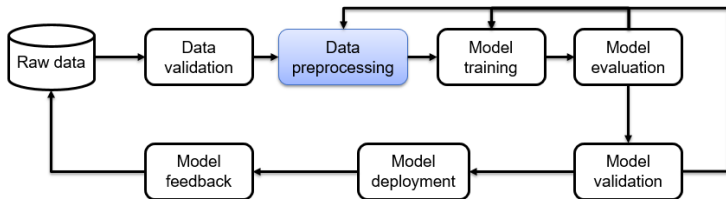


Machine Learning Pipeline



Data validation is a crucial process in data management, ensuring the **accuracy** and **quality** of data. It involves checking data against specific rules or criteria, such as data types, range constraints, and format specifications, to verify its integrity and correctness

Machine Learning Pipeline



- Data clean
- Handle missing values
- Handle outlier values
- Feature extraction and transformation
- Feature selection

Why is Machine Learning Important?



Phase 1 : Learning



Pre-Processing

- Normalization
- Dimension Reduction
- Image Processing, etc.



Learning

- Supervised
- Unsupervised
- Minimization, etc.



Error Analysis

- Precision/recall
- Over fitting
- Test/cross Validation data, etc.



Phase 2: Prediction



Model



New Data



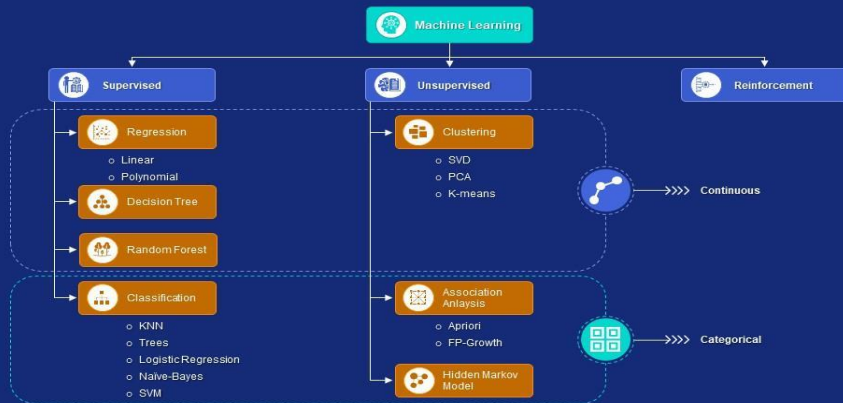
Prediction



Predicted Data

This slide is 100% editable. Adapt it to your needs and capture your audience's attention.

Machine Learning Algorithms



This slide is 100% editable. Adapt it to your needs and capture your audience's attention.

What is machine learning?

Type of learning

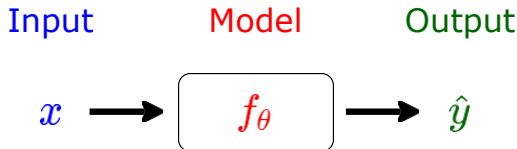
- ✓ Supervised (inductive) learning
- ✓ Unsupervised learning
- ✓ Semi-Supervised learning
- ✓ Reinforcement learning

Type of learning

- **Supervised (inductive) learning**
 - Given: training data + desired outputs (labels)
- **Unsupervised learning**
 - Given: training data (without desired outputs)
- **Semi-supervised learning**
 - Given: training data + a few desired outputs
- **Reinforcement learning**
 - Rewards from sequence of actions

What is machine learning?

- **Supervised learning.** We get observed inputs $D = \{\mathbf{x}_n : n = 1 : N\}$ and corresponding outputs y_n . We learn a mapping f from inputs to outputs.



- **Unsupervised learning.** We just get observed inputs D without any corresponding outputs. We try to make sense of data.
- **Reinforcement learning.** An **agent** has to learn how to interact with its environment. This can be encoded by a **policy** $a = \pi(\mathbf{s})$, which specifies which action a to perform in response to each possible input $\mathbf{s}$ (environmental state).

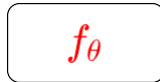
Supervised Learning - Classification

Learning a mapping $f_{\theta} : \mathbb{R}^{W \times H} \rightarrow \{\text{"cat"}, \text{"non-cat"}\}$.

Input

Model

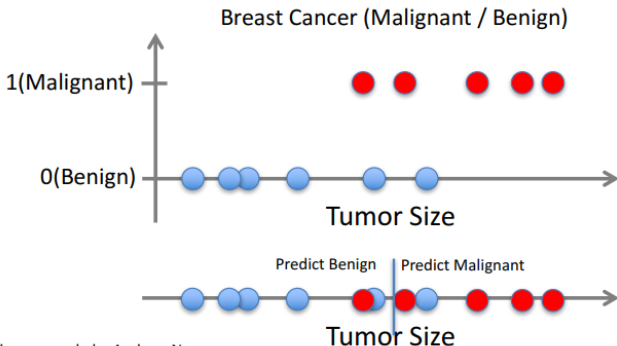
Output



"cat"

Supervised Learning - Classification

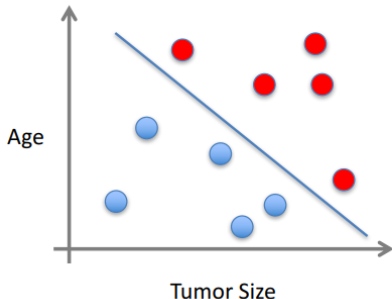
- Given $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$
- Learn a function $f(x)$ to predict y given x
 - y is categorical == classification



Based on example by Andrew Ng

Supervised Learning - Classification

- x can be multi-dimensional
 - Each dimension corresponds to an attribute



- Clump Thickness
- Uniformity of Cell Size
- Uniformity of Cell Shape
- ...

In image classification,

- The X is the set of images, which is very high-dimensional.
- For a color image with $C = 3$ channels (RGB) and $D_1 \times D_2$ pixels, we have $X = \mathbb{R}^D$, where $D = C \times D_1 \times D_2$.
- Learning a mapping $f : X \rightarrow Y$ from images to labels is challenging.

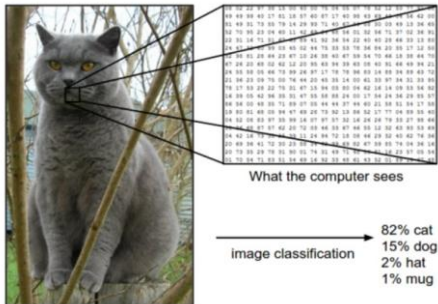


Figure: Illustration of the image classification problem.

In **classification** problems, the output space is

- a set of C unordered and mutually exclusive labels known as **classes** $Y = \{1, 2, \dots, C\}$.
- If there are just two classes, $y \in \{0, 1\}$ or $y \in \{+1, -1\}$, it is called **binary classification**.



(a)



(b)



(c)

Figure: Classifying Iris flowers into 3 subspecies: setosa, versicolor and virginica.

Classification

Machine Learning:



Human Learning:

We learn through

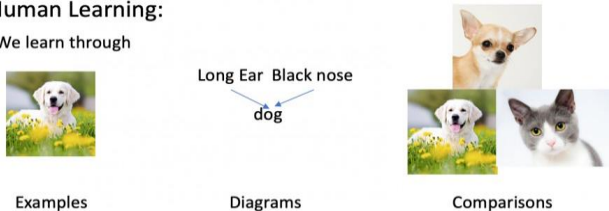
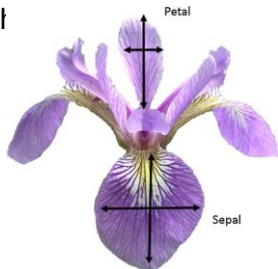


Figure: Machine learning vs. Human learning¹

¹<https://blog.ml.cmu.edu/2019/03/29/building-machine-learning-models-via-comparisons/>

The **Iris dataset** is a collection of 150 labeled examples of Iris flowers, 50 of each type, described by 4 features

- sepal length, sepal width, petal length, petal width. The input space is much lower-dimensional $X = \mathbb{R}^4$. These numeric features are high



index	sl	sw	pl	pw	label
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
...					
50	7.0	3.2	4.7	1.4	Versicolor
...					
149	5.9	3.0	5.1	1.8	Virginica

Figure: A subset of the Iris design matrix.

A small dataset of features can be stored as an $N \times D$ **design matrix**.

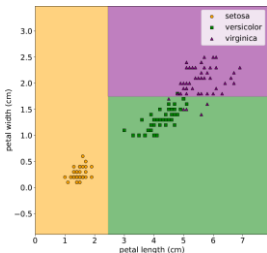


Figure: Decision surface

- We apply a **decision rule** to the Iris dataset

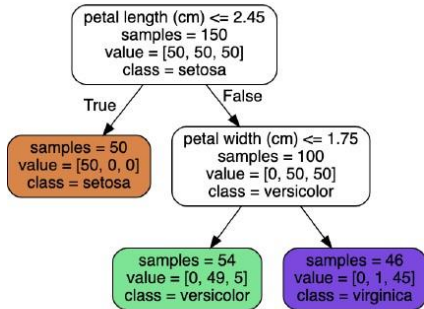
$$f_1(\mathbf{x}; \boldsymbol{\theta}) = \begin{cases} \text{Setosa if petal length} < 2.45 \\ \text{Versicolor or Virginica otherwise} \end{cases}$$

- This rule defines a 1d **decision boundary** at $x_{\text{petal length}} = 2.45$, partitioning the input space into 2 regions.
- We add another **decision rule** to inputs that fail the first rule.

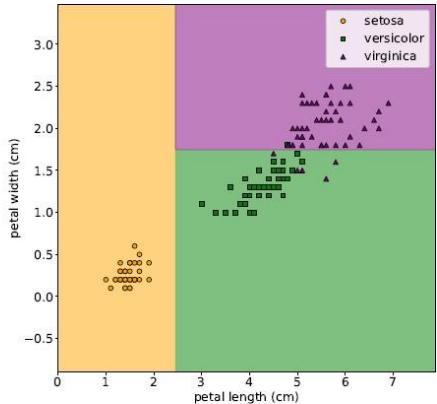
$$f_2(\mathbf{x}; \boldsymbol{\theta}) = \begin{cases} \text{Versicolor if petal width} < 1.75 \\ \text{Virginica otherwise} \end{cases}$$

- Both rules induces a 2d **decision surface**.

Learning a classifier



(a)



(b)

Figure: A decision tree on Iris dataset, using just petal length and width features.

Learning a classifier

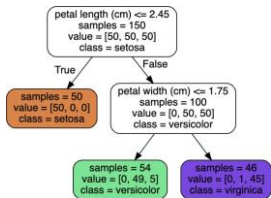


Figure: Decision tree

- We can arrange the rules f_1, f_2 into a tree structure, called a **decision tree**.
- We can represent the tree by storing, for each internal node:
 - the feature index that is used.
 - the corresponding threshold value.
- We denote all these **parameters** by θ .

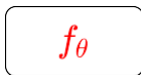
Supervised Learning - Regression

Learning a mapping $f_{\theta} : \mathbb{R}^N \rightarrow \mathbb{R}$.

Input

Model

Output



\$50,312

Regression

Suppose we want to predict a real-valued quantity $y \in \mathbb{R}$ instead of a **class label** $y \in \{1, 2, \dots, C\}$.

E.g., in the case of Iris flowers, y might be **the average height** of the plant, or **the degree of toxicity**.

This type of problem is known as **regression**.

Supervised Learning - Regression

— Example: Price of a used car

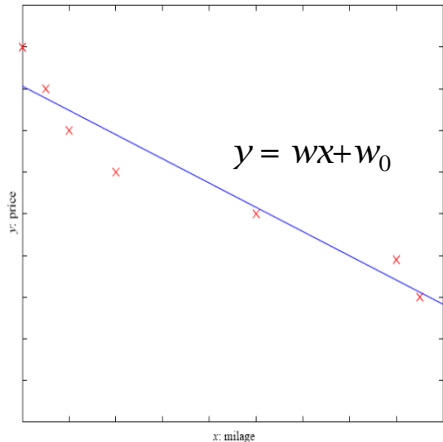
— x : car attributes

y : price

$$y = f(x | \theta)$$

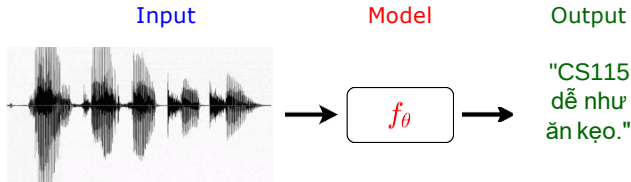
$f()$ model,

θ parameters

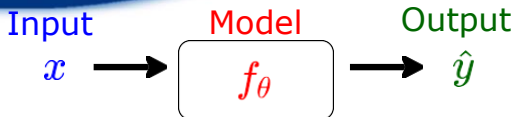


Supervised Learning - Structured Prediction

Learning a mapping $f_{\theta} : \mathbb{R}^N \rightarrow \{1, 2, \dots, L\}^M$.



Supervised learning



The most common form of ML is **supervised learning**.

- The task T is to learn a mapping f from inputs $x \in \mathcal{X}$ to outputs $y \in \mathcal{Y}$: Estimate **parameters** θ from training data $\{(\mathbf{x}_n, y_n)\}_{n=1}^N$.
- The inputs x are also called the **features**, or **predictors**.
- The output y is also known as the **label**, **target**, or **response**.
- The experience E is a set of N input-output pairs $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$, known as the **training set**. N is called the **sample size**.
- The performance measure P depends on the type of output we are predicting.

Empirical risk minimization

The goal of supervised learning is

- to automatically come up with classification models.
- to reliably predict the labels for any given input.

The **misclassification rate** on the training set can be used to measure performance on this task:

$$L(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i \neq f(\mathbf{x}_i; \boldsymbol{\theta}))$$

where $\mathbb{I}(e)$ is the binary **indicator function**:

$$\mathbb{I}(e) = \begin{cases} 1 & \text{if } e \text{ is true} \\ 0 & \text{if } e \text{ is false} \end{cases}$$

Empirical risk minimization

The problem of **model fitting** or **training** can be defined as finding a setting of parameters that minimizes the empirical risk on the training set:

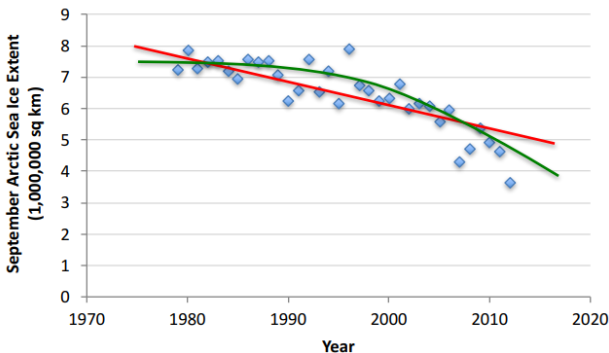
$$\hat{\theta} = \operatorname{argmin}_{\theta} L(\theta) = \operatorname{argmin}_{\theta} \frac{1}{N} \sum_{i=1}^N l(y_i, f(x_i; \theta))$$

This is called **empirical risk minimization**.

However, our true goal is:

- to minimize the expected loss on **future data** that we have not seen,
- to **generalize**, rather than just do well on the training set.

- Given $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$
- Learn a function $f(x)$ to predict y given x
 - y is real-valued == regression



Data from G. Witt. Journal of Statistics Education, Volume 21, Number 1 (2013)

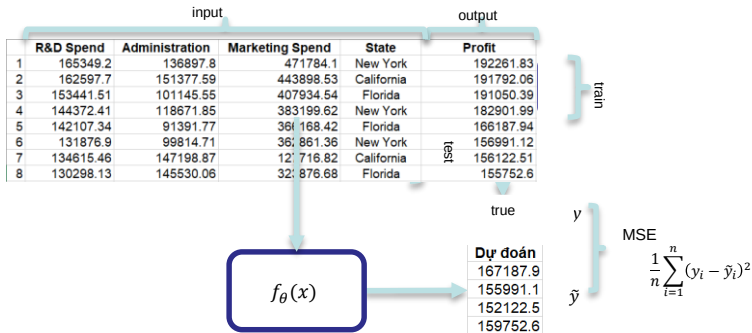
- Regression is very similar to classification, but since the output $\hat{y} = f(\mathbf{x}; \boldsymbol{\theta})$ is real-valued, **we need to use a different loss function?**
- Why don't we use the misclassification rate with the zero-one loss?

$$L(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N l_{01}(y_i, \hat{y}_i) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i \neq f(\mathbf{x}_i; \boldsymbol{\theta}))$$

- The most common choice is **quadratic loss**, or l_2 loss:
 $l_2(y, \hat{y}) = (y - \hat{y})^2$
- This penalizes large **residuals** $y - \hat{y}$ more than small ones.
- The empirical risk when using quadratic loss is equal to the **mean squared error (MSE)**:

$$L(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N (y_i - f(\mathbf{x}_i; \boldsymbol{\theta}))^2$$

Regression (loss function)



Regression: Metric

	R&D Spend	Administration	Marketing Spend	State	Profit
1	165349.2	136897.8	471784.1	New York	192261.83
2	162597.7	151377.59	443898.53	California	191792.06
3	153441.51	101145.55	407934.54	Florida	191050.39
4	144372.41	118671.85	383199.62	New York	182901.99
5	142107.34	91391.77	366168.42	Florida	166187.94
6	131876.9	99814.71	362861.36	New York	156991.12
7	134615.46	147198.87	127716.82	California	156122.51
8	130298.13	145530.06	323876.68	Florida	155752.6

train

test

True

y

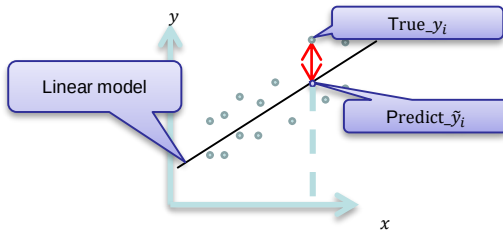
$\hat{y}$

Metric:

- MAE
- MSE
- RMSE
- ...



Regression: Metric



$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

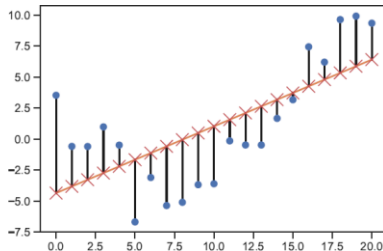
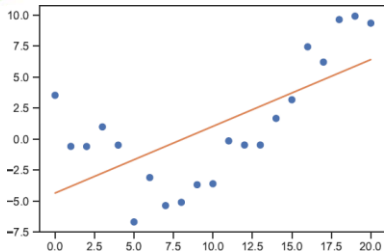
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

$$\text{RAE} = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{\sum_{i=1}^n |y_i - \bar{y}|}$$

$$\text{RSE} = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Linear Regression



We can fit the 1d data above using a **simple linear regression** model:

$$f(x; \theta) = b + wx$$

where w is the **slope**, and b is the **offset**. $\theta = (w, b)$ are the parameters of the model. By adjusting θ , we can minimize the sum of squared errors until we find the **least squares solution**:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \operatorname{MSE}(\theta)$$

If we have multiple input features, we can write

$$f(\mathbf{x}; \boldsymbol{\theta}) = b + w_1x_1 + w_2x_2 + \cdots + w_Dx_D = b + \mathbf{w}^T \mathbf{x}$$

where $\boldsymbol{\theta} = (\mathbf{w}, b)$. This is called **multiple linear regression**.

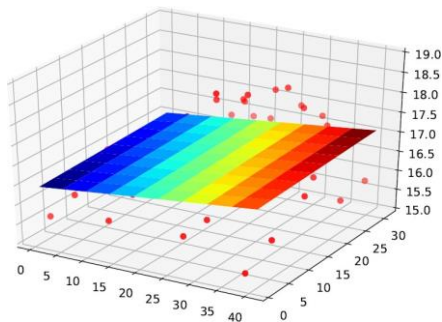
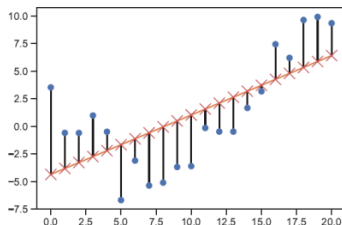
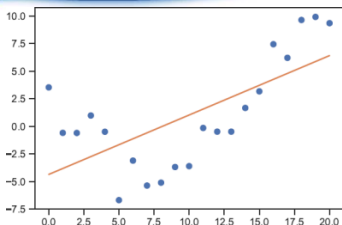


Figure: We predict the temperature as a function of 2d location in a room.

Linear model: $f(\mathbf{x}; \boldsymbol{\theta}) = b + w_1x_1 + w_2x_2$

Polynomial regression



We can improve the fit by a **polynomial regression** model of degree D :

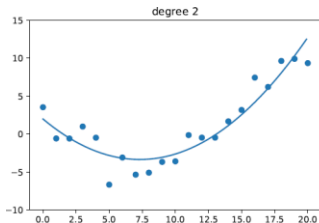
$$f(x; \mathbf{w}) = \mathbf{w}^T \varphi(x)$$

where $\varphi(x)$ is a feature vector derived from the input:

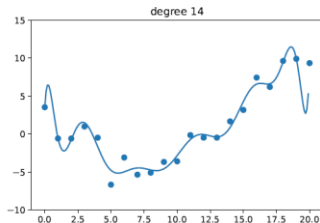
$$\varphi(x) = [1, x, x^2, \dots, x^D]$$

This is an example of **feature engineering**.

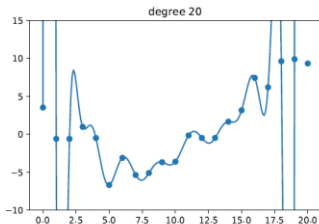
Polynomial regression



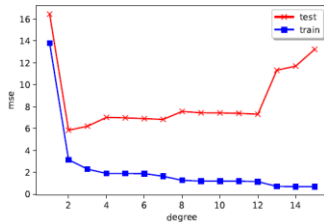
(a)



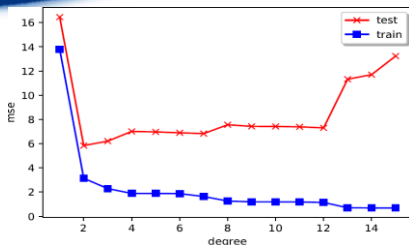
(b)



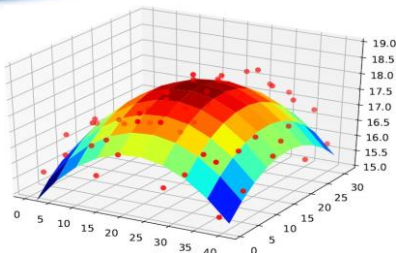
(c)



(d)



- ❑ Using $D = 2$ results in a much better fit.
- ❑ We can keep increasing D , and hence the number of parameters in the model, until $D = N - 1$.
- ❑ In this case, we have one parameter per data point, so we can perfectly **interpolate** the data. The resulting model will have 0 MSE. However, the resulting function will not be a good predictor.
- ❑ A model that perfectly fits the training data, but which is too complex, is said to suffer from **overfitting**.



We can apply polynomial regression to multi-dimensional inputs. For example, we use a quadratic expansion of the inputs as the prediction function for the temperature model

The prediction function is still a linear function of the parameters $\mathbf{w}$, even though it is a nonlinear function of the original input $\mathbf{x}$.

ML issues

Training data

- How many observation are enough for learning?
- Whether or not does the size of the training set affect performance of an ML system?
- What is the effect of the disrupted or noisy observation?

Learnability

- The goodness/limit of the learning algorithm.
- What is the generalization of the system? (Predict well new observations, not only the training data. Avoid **overfitting**).

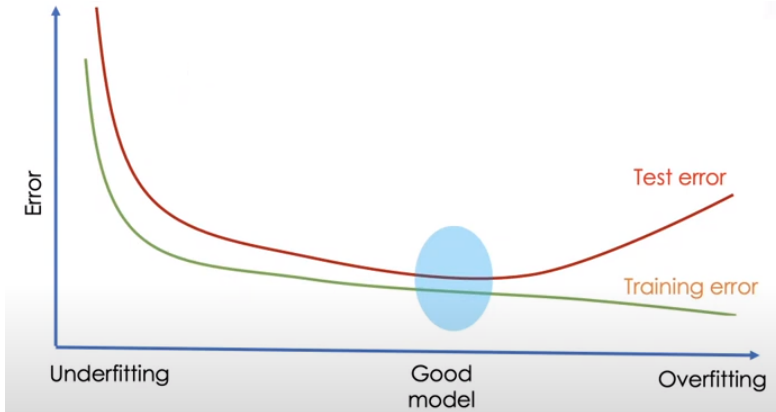
- We can rewrite the empirical risk equation as

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N l(y_i, f(x_i; \theta))$$
$$L(\theta; D_{train}) = \frac{1}{|D_{train}|} \sum_{(x,y) \in D_{train}} l(y, f(x; \theta))$$

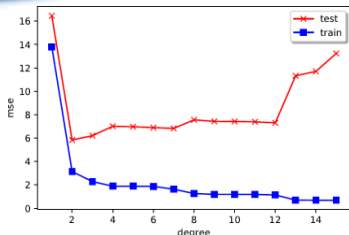
where $|D_{train}|$ is the size of the training set D_{train} .

- This makes explicit which dataset the loss is being evaluated on.
- With a **complex** enough model (or simply by memorizing all training data), we can achieve **zero** training loss.
- But we care about prediction accuracy on new data, that are not in the training set.
- A model that perfectly fits the training data, but which is too complex, is said to suffer from **overfitting**.

What is overfitting? And why?



Overfitting and Generalization



The training error goes to 0 as the model becomes more complex. The test error has a **U-shaped curve**:

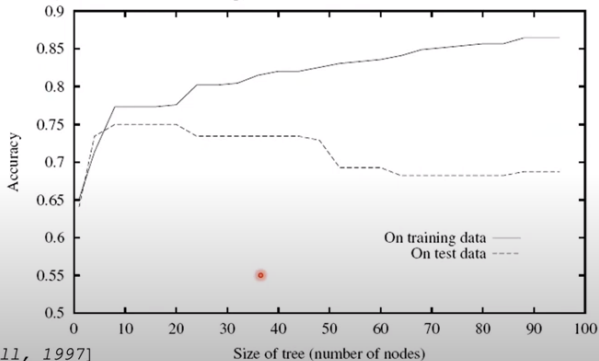
On the left, where $D = 1$, the model is **underfitting**. On the right, where $D \gg 1$, the model is **overfitting**. When $D = 2$, the model complexity is “just right”.

The most complex model has the most **degrees of freedom** $\rightarrow$ minimum training loss.

We should pick the model with the minimum test loss.

What is overfitting? And why?

- Increasing the size of a decision tree can degrade prediction on unseen data, even though increasing the accuracy for the training data.



[Mitchell, 1997]

Let $p^*(\mathbf{x}, y)$ denote the true (and **unknown**) distribution of the training data. Instead of the empirical risk, we have the **population risk** (or the **theoretical** expected loss):

$$L(\boldsymbol{\theta}; p^*) = \mathbb{E}_{p^*(x,y)}[l(y, f(\mathbf{x}; \boldsymbol{\theta}))]$$

The difference $L(\boldsymbol{\theta}; p^*) - L(\boldsymbol{\theta}; D_{train})$ is the **generalization gap**. If a model has a large generalization gap (i.e., low empirical risk but high population risk), it is a sign of overfitting.

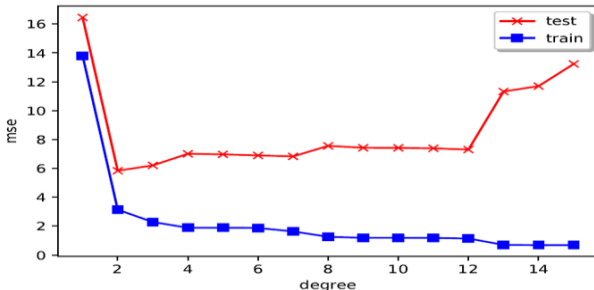
However, in practice, we don't know p^* .

We partition the data we have into two subsets: the training set and the **test set**. We approximate the population risk using the **test risk**:

$$L(\boldsymbol{\theta}; D_{test}) = \frac{1}{|D_{test}|} \sum_{(\mathbf{x}, y) \in D_{test}} l(y, f(\mathbf{x}; \boldsymbol{\theta}))$$

Overfitting and Generalization

- ❑ In practice, we partition the data into 3 sets: a training set, a test set, and a **validation set**.
- ❑ The training set is used for model fitting. The validation set is used for model selection
- ❑ The test set is used to estimate future performance (i.e., the population risk).
- ❑ The test set is NOT used for model fitting or model selection.



We can't perfectly predict the exact output given the input, due to

- lack of knowledge of the input-output mapping (**model uncertainty**)
- and/or intrinsic (irreducible) stochasticity in the mapping (**data uncertainty**).

We capture our uncertainty using **conditional probability distribution**:

$$p(y = c | \mathbf{x}; \boldsymbol{\theta}) = f_c(\mathbf{x}; \boldsymbol{\theta})$$

where $f : X \rightarrow [0, 1]^C$ maps inputs to a probability distribution over the C possible output labels.

Since $f_c(\mathbf{x}; \boldsymbol{\theta})$ returns the probability of class label c , we require

$$0 \leq f_c \leq 1 \text{ for each } c, \text{ and } \sum_{c=1}^C f_c = 1$$

To avoid this restriction, it is common to require the model to return unnormalized log-probabilities. We can then use the **softmax function** to convert these to probabilities

$$S(\mathbf{a}) = \left[\frac{e^{a_1}}{\sum_{k=1}^C e^{a_k}}, \frac{e^{a_2}}{\sum_{k=1}^C e^{a_k}}, \dots, \frac{e^{a_C}}{\sum_{k=1}^C e^{a_k}} \right]$$

$$S : \mathbb{R}^C \rightarrow [0, 1]^C, \text{ and satisfied } 0 \leq S(\mathbf{a}) \leq 1 \text{ and } \sum_{k=1}^C S(\mathbf{a})_k = 1$$

The inputs to the softmax, $\mathbf{a} = f(\mathbf{x}; \boldsymbol{\theta})$, are called **logits**.

$$p(y = c | \mathbf{x}; \boldsymbol{\theta}) = S_c(f(\mathbf{x}; \boldsymbol{\theta}))$$

The overall model is defined as

$$p(y = c|\mathbf{x}; \boldsymbol{\theta}) = S_c(f(\mathbf{x}; \boldsymbol{\theta}))$$

A special case is when f is an **affine function**

$$f(\mathbf{x}; \boldsymbol{\theta}) = b + \mathbf{w}^T \mathbf{x} = b + w_1 x_1 + w_2 x_2 + \dots + w_D x_D$$

where $\boldsymbol{\theta} = (b, \mathbf{w})$ are the parameters of the model.

$\mathbf{w}$ are called the **weights** and b is called the **bias**. This model is called **logistic regression**.

We can define and so that $\tilde{\mathbf{w}} = [b, w_1, \dots, w_D]$ and $\tilde{\mathbf{x}} = [1, x_1, \dots, x_D]$

$$\tilde{\mathbf{w}}^T \tilde{\mathbf{x}} = b + \mathbf{w}^T \mathbf{x}$$

This converts the affine function into a **linear function**. For simplicity, we can just write the prediction function as

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{w}^T \mathbf{x}$$

- When fitting probabilistic models, we can use the **negative log probability** as our loss function:

$$l(y, f(x; \theta)) = -\log p(y | f(x; \theta))$$

- A good model is one that assigns a high probability to the true output y for each corresponding input x
- The average negative log probability of the training set is given by

$$\text{NLL}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log p(y_i | f(x_i; \theta))$$

- This is called the **negative log likelihood**.
- If we minimize this $\hat{\theta}_{MLE}$, we can compute the **maximum likelihood estimate** (MLE):

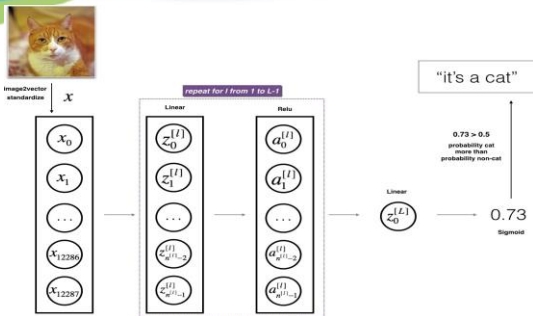
$$\hat{\theta}_{MLE} = \underset{\theta}{\operatorname{argmin}} \text{NLL}(\theta)$$

- Previously, we **manually** specify the transformation of the input features (feature engineering). For example, in polynomial regression:

$$\phi(x) = [1, x, x^2, x^3, \dots, x^D]$$

- We can create more powerful models by learning to do such nonlinear **feature extraction** **automatically**.
- We let $\phi(x)$ have its own set of parameters V :
$$f(x; w, V) = w^T \phi(x; V)$$

Deep neural networks

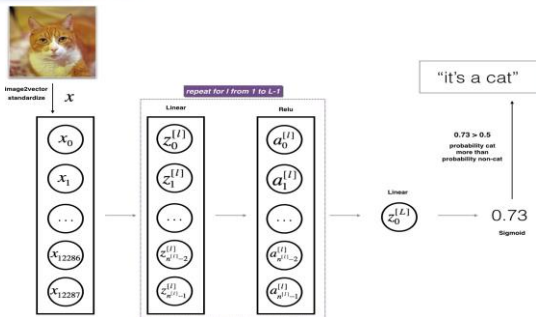


The **feature extractor** $\phi(x; V)$ is a composition of simpler functions. The resulting model is a stack of L nested functions:

$$f(x; \theta) = f(x; w, V) = f_L(f_{L-1}(\dots (f_1(x)) \dots))$$

where $f_l(f_{1:l-1}(x)) = f(f_{1:l-1}(x); \theta_l)$ is the function at layer l .

Deep neural networks



$$f(x; w, V) = w^T \phi(x; V)$$

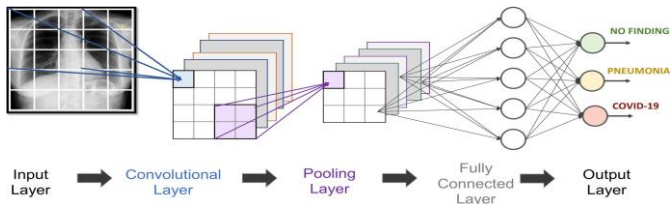
Typically, the final layer L is *linear* and has the form:

$$f_L(f_{1:L-1}(x)) = w^T f_{1:L-1}(x) = w^T \phi(x; V)$$

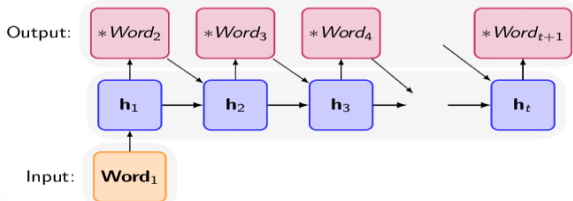
where $f_{1:L-1}(x)$ is the **learned** feature extractor $\phi(x; V)$

Common DNN variants are:

- Convolutional neural networks (CNNs) for images.



- Recurrent neural networks (RNNs) for sequences.



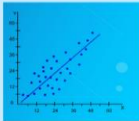


REGRESSION

Regression analysis is a powerful statistical method that allows you to examine the relationship between two or more variables of interest.



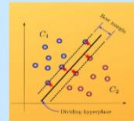
Types of regression



Linear Regression



Polynomial Regression



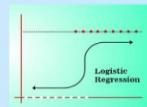
Super Vector Regression



Decision Tree Regression



Random Forest Regression



Logistic Regression